

KCPC × KCL WiSTEM

Solution Sheet

Logic & Lattes

Developed by the KCPC Internal Committee, this solution sheet accompanies the joint *KCL WiSTEM* social workshop problem sheet.

I. Logical Problems — Solutions

1. Left-Handed People

Solution. 50 people must leave.

Explanation. Initially there are 99 left-handed people and 1 right-handed person. To make the single right-handed person represent 2% of the total population, let the new population size be x . Then

$$0.02x = 1$$

which implies

$$x = 50.$$

Thus the new total number of people in the room must be 50. Since the original number of people was 100, we must have

$$100 - 50 = 50$$

people leaving the room.

2. The Three Wise Men

Solution. The man announced that he was wearing a **blue** hat.

Explanation. The king guaranteed that at least one hat is blue.

If there were only one blue hat, that person would see two white hats and would immediately know that his own hat must be blue. Since nobody answered immediately, everyone could deduce that there was not exactly one blue hat.

If there were exactly two blue hats, each blue-hat wearer would see one blue hat and one white hat. Each would wait to see if the other person declared their hat colour immediately (as would happen if there were only one blue hat). When neither person answered quickly, each would deduce that the other must also see a blue hat.

Because the men sat in silence for a long time, the eventual speaker realised that the situation could not involve only one or two blue hats. Therefore, all three hats must be blue, and he concluded that his own hat was blue as well.

3. Ants on a Ruler

Key Insight. When two ants moving at the same speed collide and reverse direction, this is mathematically equivalent to the ants passing through each other.

Solution. Since the ants are indistinguishable and all move with constant speed v , we may treat collisions as if the ants simply pass through one another.

In the worst-case scenario, an ant begins at one end of the ruler and must travel the entire length L before falling off.

Therefore the maximum possible time before all ants have fallen off the ruler is

$$T = \frac{L}{v}.$$

4. Simple Stick Game

Solution. A *winning state* is a position from which the current player can force a win assuming optimal play. A *losing state* is a position from which the current player will lose if the opponent plays optimally.

State 0 is clearly a losing state, since the player cannot make any move. States 1, 2, and 3 are winning states, because the player can remove 1, 2, or 3 sticks respectively and immediately win the game.

State 4 is therefore a losing state, because any move from 4 leads to one of the states 1, 2, or 3, which are winning states for the opponent.

More generally, a state is winning if there exists a move that leads to a losing state. Otherwise, the state is losing.

Using this observation, we can classify the positions of the game. A state with k sticks is a losing state exactly when

$$k \equiv 0 \pmod{4}.$$

Otherwise, the state is winning.

Optimal Strategy. The optimal strategy is to always remove a number of sticks such that the number remaining in the heap is divisible by 4. Repeating this strategy eventually forces the opponent into state 0, meaning they cannot make a move and therefore lose the game.

5. Highly Divisible Triangular Number

Answer. The first triangular number with more than 500 divisors is

$$76576500.$$

Explanation.

The n th triangular number is defined as

$$T_n = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

Observe that n and $n+1$ are consecutive integers and therefore

$$\gcd(n, n+1) = 1.$$

Hence the two terms are coprime.

Exactly one of them must be even, so we can rewrite the triangular number as the product of two coprime integers:

$$T_n = \begin{cases} \left(\frac{n}{2}\right)(n+1) & \text{if } n \text{ is even} \\ n\left(\frac{n+1}{2}\right) & \text{if } n \text{ is odd} \end{cases}$$

For two coprime integers a and b , the number of divisors satisfies

$$d(ab) = d(a)d(b)$$

where $d(x)$ denotes the number of positive divisors of x .

Divisor counting using prime factorisation.

If

$$x = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$$

then the number of divisors of x is

$$d(x) = (e_1 + 1)(e_2 + 1) \cdots (e_k + 1).$$

This follows because each divisor may include each prime factor with exponent

$$0, 1, 2, \dots, e_i.$$

Optimisation idea.

Instead of factoring the entire triangular number every time, we factor the two smaller coprime terms.

Furthermore, when moving from T_n to T_{n+1} , one of the factors appears again. Therefore we only need to factor one new number at each step and reuse the previously computed divisor count for the other factor.

This significantly reduces the computational cost.

Pseudocode.

```
function divisor_count(x):
    total <- 1
    exponent <- 0

    while x is divisible by 2:
        x <- x / 2
        exponent <- exponent + 1

    if exponent > 0:
        total <- total * (exponent + 1)

    p <- 3

    while p * p <= x:
        exponent <- 0
        while x is divisible by p:
            x <- x / p
            exponent <- exponent + 1

        if exponent > 0:
            total <- total * (exponent + 1)
```

```

        p <- p + 2

    if x > 1:
        total <- total * 2

    return total

function first_triangle_with_over_k_divisors(k):
    n <- 1

    while true:
        if n is even:
            a <- n / 2
            b <- n + 1
        else:
            a <- n
            b <- (n + 1) / 2

        total_divisors <- divisor_count(a) * divisor_count(b)

        if total_divisors > k:
            return n * (n + 1) / 2

        n <- n + 1

```

Python Implementation.

```

def divisor_count(n: int) -> int:
    total = 1

    exponent = 0
    while n % 2 == 0:
        n //= 2
        exponent += 1
    if exponent > 0:
        total *= (exponent + 1)

    p = 3
    while p * p <= n:
        exponent = 0
        while n % p == 0:
            n //= p
            exponent += 1
        if exponent > 0:
            total *= (exponent + 1)
        p += 2

    if n > 1:
        total *= 2

```

```

    return total

def first_triangle_with_over_k_divisors(k: int) -> int:
    n = 1

    while True:
        if n % 2 == 0:
            a = n // 2
            b = n + 1
        else:
            a = n
            b = (n + 1) // 2

        total_divisors = divisor_count(a) * divisor_count(b)

        if total_divisors > k:
            return n * (n + 1) // 2

        n += 1

if __name__ == "__main__":
    print(first_triangle_with_over_k_divisors(500))

```

Why this works.

The algorithm relies on three facts:

- The n th triangular number is $T_n = \frac{n(n+1)}{2}$.
- Consecutive integers are coprime.
- The divisor function is multiplicative over coprime integers.

By decomposing the triangular number into two coprime factors and reusing one divisor count between iterations, we avoid repeatedly factoring very large numbers.

The algorithm therefore finds the first triangular number with more than 500 divisors efficiently.

Important Property. If a and b are coprime, then the number of divisors satisfies

$$d(ab) = d(a)d(b),$$

where $d(x)$ denotes the number of positive divisors of x .

Definition (Coprime Numbers). Two integers a and b are said to be *coprime* (or *relatively prime*) if their greatest common divisor is 1:

$$\gcd(a, b) = 1.$$

Equivalently, a and b share no common prime factors.

For example, 8 and 15 are coprime because

$$8 = 2^3, \quad 15 = 3 \cdot 5$$

and they have no prime factors in common.

6. Card Sums Puzzle

Answer. C. Romy has card 5.

Solution.

Each friend receives two distinct cards from the set $\{1, 2, \dots, 12\}$.

The possible pairs of cards corresponding to each declared total are:

Name (Total)	Possible Card Pairs
Paolo (4)	(1, 3)
Quinn (11)	(1, 10), (2, 9), (3, 8), (4, 7), (5, 6)
Romy (16)	(4, 12), (5, 11), (6, 10), (7, 9)
Stephen (19)	(7, 12), (8, 11), (9, 10)
Thomas (20)	(8, 12), (9, 11)

The sum of all cards from 1 to 12 is

$$1 + 2 + \dots + 12 = 78.$$

The totals declared by the five friends sum to

$$4 + 11 + 16 + 19 + 20 = 70.$$

Therefore the two unused cards must sum to

$$78 - 70 = 8.$$

Paolo's total is 4, and the only possible pair is

$$(1, 3).$$

Thus Paolo holds cards 1 and 3.

Removing these cards leaves the unused pair summing to 8, which must therefore be

$$(2, 6).$$

Now consider Thomas, whose total is 20. The only possible pairs are

$$(12, 8) \quad \text{or} \quad (11, 9).$$

Suppose Thomas had (11, 9). Then Stephen must take (12, 7) to make 19. However, with these cards removed, there would be no remaining pair that sums to 16 for Romy. Hence this case is impossible.

Therefore Thomas must hold

$$(12, 8).$$

Removing these cards leaves the remaining assignments:

Stephen : (10, 9)

Romy : (11, 5)

Quinn : (7, 4).

Thus the final card assignments are

Name (Total)	Cards
Paolo (4)	(1, 3)
Quinn (11)	(7, 4)
Romy (16)	(11, 5)
Stephen (19)	(10, 9)
Thomas (20)	(12, 8)

Among the options provided, the only true statement is

C: Romy has card 5.

7. Aardvarks and Anteaters

Answer. B. Each animal eats 66 ants.

Solution.

There are

$$n + 12$$

animals in total. Since each animal eats the same whole number of ants, let that number be m . Then

$$m = \frac{n^2 + 20n + 25}{n + 12}.$$

We now simplify this expression by division:

$$\frac{n^2 + 20n + 25}{n + 12} = \frac{(n + 12)(n + 8) - 71}{n + 12} = n + 8 - \frac{71}{n + 12}.$$

So

$$m = n + 8 - \frac{71}{n + 12}.$$

Since m must be an integer, it follows that

$$\frac{71}{n + 12}$$

must also be an integer. Therefore $n + 12$ must be a factor of 71.

Now 71 is prime, so its only positive factors are

$$1 \quad \text{and} \quad 71.$$

Since n is a positive integer, we have

$$n + 12 > 1.$$

Hence the only possible value is

$$n + 12 = 71.$$

Therefore

$$n = 59.$$

Substituting back:

$$m = 59 + 8 - \frac{71}{71} = 67 - 1 = 66.$$

Thus each animal eats

$$\boxed{66}$$

ants.

8. Shaded Squares

Answer. C. The difference is 11.

Solution.

Number the squares from 1 to 50 from left to right.

Largest possible number of shaded squares.

By rule (a), no two adjacent squares may both be shaded. Therefore, from each pair of consecutive squares, at most one can be shaded. Hence at most half of the 50 squares can be shaded, so the maximum possible number is

$$25.$$

This bound can be achieved by shading all odd-numbered squares from 1 to 47, together with square 50:

$$1, 3, 5, \dots, 47, 50.$$

Note that square 49 cannot be shaded, since square 50 is already shaded. This gives a total of 25 shaded squares.

Smallest possible number of shaded squares.

By rule (b), there may not be four consecutive unshaded squares. Therefore, in every block of four consecutive squares, at least one square must be shaded.

Among the squares from 2 to 49, there are 48 squares. To ensure that there are never four consecutive unshaded squares, we need at least one shaded square in each group of four. Hence at least

$$\frac{48}{4} = 12$$

of the squares from 2 to 49 must be shaded.

Since squares 1 and 50 are also required to be shaded, the minimum possible total number of shaded squares is therefore

$$12 + 2 = 14.$$

This bound can be achieved by shading every fourth square starting from square 1 up to square 45, and then shading square 47 as well as square 50:

$$1, 5, 9, \dots, 45, 47, 50.$$

This gives exactly 14 shaded squares, and satisfies both conditions.

Therefore:

$$\text{maximum} = 25, \quad \text{minimum} = 14.$$

So the required difference is

$$25 - 14 = 11.$$

Thus the correct answer is

$$\boxed{11}.$$

II. Programming Problems — Solutions

1. LeetCode 11 — Container With Most Water

Solution Outline.

We are given an array `height` where the value at index i represents the height of a vertical line drawn at that position. Two lines together with the x -axis form a container that can hold water.

If we choose two indices $i < j$, the amount of water that can be contained between them is

$$\text{Area} = (j - i) \cdot \min(\text{height}[i], \text{height}[j]).$$

The width of the container is the horizontal distance $(j - i)$, while the height of the water is limited by the shorter of the two vertical lines.

Naive approach.

A straightforward approach would be to examine every possible pair of indices (i, j) and compute the area. However this requires checking approximately

$$\frac{n(n-1)}{2}$$

pairs, resulting in a time complexity of

$$O(n^2),$$

which is inefficient for large inputs.

Key idea.

Instead we use a **two-pointer strategy**.

- Place one pointer at the beginning of the array ($left = 0$).
- Place the second pointer at the end of the array ($right = n - 1$).
- This pair gives the *maximum possible width*.

At each step we compute the container area

$$(right - left) \cdot \min(\text{height}[left], \text{height}[right]).$$

We update the maximum area found so far.

The crucial observation is that the **shorter line limits the water level**. Therefore:

- If $\text{height}[left] < \text{height}[right]$, the left line is the limiting factor. Moving the right pointer cannot increase the area because the width decreases while the limiting height remains unchanged. Thus we increment $left$.
- Otherwise, we decrement $right$.

This allows us to discard positions that cannot lead to a better solution.

Why this works.

Suppose $height[left] \leq height[right]$. The current area is

$$A = (right - left) \cdot height[left].$$

If we keep $left$ fixed and move $right$ inward, the width decreases and the minimum height cannot exceed $height[left]$. Therefore the resulting area must be strictly smaller. Hence index $left$ cannot produce a better container and may safely be discarded.

A symmetric argument holds when $height[right] < height[left]$.

Algorithm.

- (a) Initialise $left = 0$ and $right = n - 1$.
- (b) While $left < right$:
 - Compute the current container area.
 - Update the maximum area.
 - Move the pointer corresponding to the smaller height inward.
- (c) When the pointers meet, return the maximum area found.

Pseudocode.

```
left = 0
right = n - 1
max_area = 0

while left < right:
    width = right - left
    height = min(height[left], height[right])
    area = width * height
    max_area = max(max_area, area)

    if height[left] < height[right]:
        left += 1
    else:
        right -= 1

return max_area
```

Python implementation.

```
class Solution:
    def maxArea(self, height: list[int]) -> int:
        left = 0
        right = len(height) - 1
        max_area = 0

        while left < right:
            max_area = max(
                max_area,
                (right - left) * min(height[left], height[right])
            )
            if height[left] < height[right]:
                left += 1
            else:
                right -= 1
```

```

    )

    if height[left] < height[right]:
        left += 1
    else:
        right -= 1

    return max_area

```

Complexity.

- Time complexity: $O(n)$ because each pointer moves at most n times.
- Space complexity: $O(1)$ since only constant additional memory is used.

2. LeetCode 7 — Reverse Integer

Solution Outline.

We are given a signed 32-bit integer x and must return the integer obtained by reversing its digits. If the reversed integer falls outside the range of a 32-bit signed integer

$$[-2^{31}, 2^{31} - 1],$$

then we must return 0.

Key idea.

To reverse an integer numerically, we repeatedly extract the final digit and append it to a new number.

For any integer x :

$$\text{last digit} = x \bmod 10$$

and removing the last digit can be achieved by integer division:

$$x = \left\lfloor \frac{x}{10} \right\rfloor.$$

We construct the reversed number incrementally. If the current reversed value is res and the next digit is d , then the updated value is

$$res = res \times 10 + d.$$

Handling negative numbers.

If the original number is negative, we first record the sign and work with its absolute value. After reversing the digits, we reapply the negative sign.

Overflow constraint.

Since the reversed integer must remain within the 32-bit signed integer range, we must check for overflow before appending each new digit.

The maximum positive value allowed is

$$2^{31} - 1 = 2147483647.$$

Before computing

$$res = res \times 10 + d,$$

we verify that

$$res \leq \frac{2^{31} - 1}{10}.$$

If this condition fails, adding another digit would produce an overflow, and the correct output is 0.

Algorithm.

- (a) Record whether the number is negative.
- (b) Convert the number to its absolute value.
- (c) Initialise $res = 0$.
- (d) While $x > 0$:
 - Extract the last digit $d = x \bmod 10$.
 - Remove the digit from x .
 - Check whether appending d would cause overflow.
 - Update $res = res \times 10 + d$.
- (e) Reapply the sign if the original number was negative.
- (f) Return the result.

Pseudocode.

```
is_negative = false
if x < 0:
    is_negative = true
    x = -x

res = 0
limit = 2^31 - 1

while x > 0:
    digit = x % 10
    x = x // 10

    if res > limit // 10 or (res == limit // 10 and digit > 7):
        return 0

    res = res * 10 + digit

if is_negative:
    res = -res

return res
```

Python implementation.

```
class Solution:
    def reverse(self, x: int) -> int:
        negative = x < 0
        x = abs(x)
```

```

res = 0
limit = 2**31 - 1

while x > 0:
    digit = x % 10
    x //= 10

    if res > limit // 10 or (res == limit // 10 and digit > 7):
        return 0

    res = res * 10 + digit

return -res if negative else res

```

Example.

Consider

$$x = 123.$$

$123 \bmod 10 = 3$	$res = 3$
$12 \bmod 10 = 2$	$res = 32$
$1 \bmod 10 = 1$	$res = 321$

Thus the reversed integer is 321.

Complexity.

- Time complexity: $O(d)$ where d is the number of digits in x .
- Space complexity: $O(1)$.

3. LeetCode 13 — Roman to Integer

Solution Outline.

Roman numerals use seven symbols:

Symbol	Value
<i>I</i>	1
<i>V</i>	5
<i>X</i>	10
<i>L</i>	50
<i>C</i>	100
<i>D</i>	500
<i>M</i>	1000

Roman numbers are typically written from largest to smallest from left to right. In this case, the value of the number is simply the sum of all symbols.

For example:

$$XXVII = 10 + 10 + 5 + 1 + 1 = 27$$

However, Roman numerals also use a **subtractive rule**. When a smaller numeral appears before a larger one, it indicates subtraction rather than addition.

The valid subtractive cases are:

$$\begin{aligned}IV &= 4 \\IX &= 9 \\XL &= 40 \\XC &= 90 \\CD &= 400 \\CM &= 900\end{aligned}$$

Thus the key observation is:

- If a symbol is followed by a symbol with *greater value*, it should be **subtracted**.
- Otherwise, it should be **added**.

Key idea.

We process the string from left to right. At each step we compare the current character with the next character:

- If the current value is smaller than the next value, we subtract it.
- Otherwise, we add it.

The final character is always added because there is no following character to compare it with.

Example.

Consider the Roman numeral

MCMXCIV.

Processing step-by-step:

$$\begin{aligned}M &= 1000 \\CM &= 900 \\XC &= 90 \\IV &= 4\end{aligned}$$

Thus the result is

$$1000 + 900 + 90 + 4 = 1994.$$

Algorithm.

- Create a mapping from Roman symbols to integers.
- Initialise a result variable $res = 0$.
- Traverse the string from left to right.
- For each symbol:
 - If the next symbol has a larger value, subtract the current value.
 - Otherwise, add the current value.
- Return the accumulated result.

Pseudocode.

```
roman = {I:1, V:5, X:10, L:50, C:100, D:500, M:1000}
```

```

res = 0

for i from 0 to length(s)-2:
    if roman[s[i]] < roman[s[i+1]]:
        res -= roman[s[i]]
    else:
        res += roman[s[i]]

res += roman[s[last]]
return res

```

Python implementation.

```

class Solution:
    def romanToInt(self, s: str) -> int:
        roman = {
            'I': 1,
            'V': 5,
            'X': 10,
            'L': 50,
            'C': 100,
            'D': 500,
            'M': 1000
        }

        result = 0

        for a, b in zip(s, s[1:]):
            if roman[a] < roman[b]:
                result -= roman[a]
            else:
                result += roman[a]

        return result + roman[s[-1]]

```

Complexity.

- Time complexity: $O(n)$ where n is the length of the Roman numeral string.
- Space complexity: $O(1)$ since the mapping of Roman symbols contains a constant number of entries.

4. LeetCode 258 — Add Digits

Solution Outline.

We are given a non-negative integer *num*. The task is to repeatedly sum its digits until the result becomes a single-digit number.

For example:

$$38 \rightarrow 3 + 8 = 11$$

$$11 \rightarrow 1 + 1 = 2$$

Thus the final result is 2.

Naive approach.

A direct simulation repeatedly extracts and sums the digits of the number.

For example:

$$num = 38$$

$$38 = 3 + 8 = 11$$

$$11 = 1 + 1 = 2$$

This continues until the number has only one digit. Although correct, this method requires repeated digit extraction and therefore runs in $O(d)$ time, where d is the number of digits.

Key mathematical insight.

This process is known as computing the **digital root** of a number.

The digital root of a positive integer satisfies the property:

$$dr(n) = \begin{cases} 0 & \text{if } n = 0 \\ 9 & \text{if } n \equiv 0 \pmod{9} \\ n \bmod 9 & \text{otherwise} \end{cases}$$

This works because the sum of the digits of a number preserves its value modulo 9. Repeatedly summing digits therefore produces the same result as computing the number modulo 9, except that multiples of 9 map to 9 rather than 0.

Example.

Consider

$$num = 38.$$

$$38 \bmod 9 = 2.$$

Thus the digital root is 2, which matches the repeated digit-sum process.

Algorithm.

- (a) If $num = 0$, return 0.
- (b) If num is divisible by 9, return 9.
- (c) Otherwise return $num \bmod 9$.

This avoids loops entirely and runs in constant time.

Pseudocode.

```
if num == 0:
    return 0

if num mod 9 == 0:
    return 9

return num mod 9
```

Python implementation.


```
class Solution:
    def addDigits(self, num: int) -> int:
        if num == 0:
            return 0
        if num % 9 == 0:
            return 9
        return num % 9
```

Complexity.

- Time complexity: $O(1)$.
 - Space complexity: $O(1)$.
-

III. Extended Problems — Solution Notes

Important. These problems are intended as stretch challenges and explore more advanced techniques.

For this section we have intentionally not included written solutions. These problems are designed to encourage discussion, experimentation, and collaborative problem solving.

If you would like guidance, hints, or help thinking through an approach, we would be very happy to assist. Feel free to reach out through the KCPC Discord or WhatsApp channels where members of the Competitive Programmes Department can provide pointers in the right direction.